Copilot Agent Governance & Operating Model (Declarative Agents — Build, Approve, Share, Retire)

Implementation Solution Guide

Version 1.0.0 · 2026-07-24

Contents

Executive Summary 5

The situation . . . 5

What this solution delivers . . . 5

How it connects to what you already own . . . 5

Why run it yourself . . . 6

The outcome . . . 6

Solution Overview 6

The problem this solves . . . 6

What this solution is NOT . . . 7

The seven components of the operating model . . . 7

How declarative agents are built, shared, approved, and managed (verified against Microsoft Learn) 9

How this governance plugs into your existing Copilot controls (without re-teaching them) . . . 10

The standard deployment model . . . 11

Limitations and honest scope boundaries . . . 11

A note for Government tenants (GCC / GCC High / DoD) . . . 12

Requirements 12

R1. Accountable executive sponsor . . . 13

R2. Microsoft 365 Copilot licensing (and awareness of the agent billing models) . . . 13

R3. The AI Administrator role (least-privilege agent governance) . . . 13

R4. Entra group-management rights . . . 14

R5. A system of record for the governance artefacts . . . 14

R6. Known Copilot data-governance and oversharing posture (prerequisite solutions) . . . 14

R7. Change-management interface (CAB), if one exists . . . 15

R8. Nominated role-holders . . . 15

R9. Microsoft Graph PowerShell (for the governance scripts) . . . 15

R10. Understanding of the agent build surfaces in scope . . . 16

Global Requirements — Copilot (category-level) 16

G1. Microsoft 365 tenant with supported workloads . . . 16

G2. Microsoft 365 Copilot licensing . . . 17

G3. Microsoft Entra ID identity and MFA . . . 17

G4. Data readiness — permissions & oversharing controls . . . 17

G5. Information protection baseline (sensitivity labels) . . . 17

G6. Administrative roles . . . 17

G7. Network and client readiness . . . 17

G8. Governance & responsible-use policy . . . 18

Implementation 18
Step 1 — Baseline the current agent estate and its sharing posture . . . 18
Step 2 — Define the four accountabilities (the RACI) . . . 19
Step 3 — Create the Agent Register, Decision Record, and Retirement Log . . . 21
Step 4 — Create the governance security groups in Entra ID . . . 23
Step 5 — Set the enforcement controls in the Copilot Control System . . . 24
Step 6 — Publish the Agent Standards document . . . 25
Step 7 — Stand up the intake workflow . . . 26
Step 8 — Stand up the Agent Review Board and change classification . . . 27
Step 9 — Set the sharing policy and stand up the shared-agent access review . . . 29
Step 10 — Define the lifecycle and retirement process . . . 30
Step 11 — Stamp governance onto the existing estate . . . 31
Step 12 — Pilot the model for two full board cycles . . . 31
Step 13 — Expand to the whole tenant and enter steady state . . . 31
Testing and Validation 32
Gate 1 — Build eligibility is enforced . . . 33
Gate 2 — Org-wide sharing is restricted . . . 33
Gate 3 — The intake / approval gate holds . . . 33
Gate 4 — The org-catalog publish path requires admin approval . . . 34
Gate 5 — Sensitive-data agents require a data review . . . 34
Gate 6 — Broadly-shared and ownerless agents are surfaced . . . 34
Gate 7 — Retirement is real and evidenced . . . 35
Gate 8 — The governance scorecard is trending the right way . . . 35
Exit criteria for the pilot . . . 35
Troubleshooting 35
Organisational failure modes . . . 36
Technical failure modes . . . 36
Best Practices 38
Govern the sharing, not the building . . . 38
Blocking is free — use it early, unblock later . . . 39
Never delete as the first action . . . 39
Reassign before you orphan . . . 39
Make the register the source of truth for *accountability*, not inventory . . . 39
Tie the data review to the data, and reuse the Purview controls you already have . . . 39
Keep the ARB fast and cheap . . . 39
Watch the "Agents at risk" signals — but don't rebuild those platforms . . . 40

Treat review intervals shorter for agents than for apps . . . 40
Announce the gate from the business, not from IT . . . 40
Government tenants: verify feature availability before you promise it . . . 40
Deliverables Summary 40
Documentation . . . 40
Governance artefacts (pre-named, ready to create) . . . 41
PowerShell scripts (Microsoft Graph, read-only, cloud-aware) . . . 42
Scope boundary (what is intentionally delivered elsewhere) . . . 42
Appendix A — PowerShell Scripts 44

Executive Summary

The situation

Your organisation has given business users the ability to build their own AI agents. Inside Microsoft 365 Copilot, a non-technical employee can use **Agent Builder** to create a declarative agent in minutes — point it at SharePoint sites, Teams chats, or public websites for knowledge, and then share it with a colleague, a team, or the **entire organisation**, with no code and, in most tenants, with no visibility for IT or Security.

That capability is a genuine productivity gain. It is also an ungoverned surface. Agents built this way accumulate as **ownerless** (the creator left), **unreviewed** (shared more widely than intended, over data nobody assessed), and **shadow** (running with no registry entry and no accountable owner). The platform's own approval gates help for Copilot Studio agents — but a declarative **Agent Builder** agent shared with *"Anyone in your organization"* can reach the whole tenant **with no approval at all**. That specific gap is what this solution closes.

What this solution delivers

This is a **governance operating model** — the organisational policy layer for business-process agents. It is deliberately *not* a technical export tool and *not* a data-governance product; it defines **who may build a business agent, on what basis it is allowed into business use, who it may be shared with, who is accountable afterwards, and when it is retired** — and it expresses each of those decisions as a concrete, enforced control in the Microsoft 365 admin center.

It provides, ready to adopt:

- A **four-role accountability model** (Agent Owner, Business Sponsor, Data & Security Reviewer, Agent

Admin) with a RACI matrix and escalation rules.

- An **intake and approval gate** and a standing **Agent Review Board**, closing the org-wide-sharing

gap.

- A published **standards document** (data-and-sharing review criteria, naming, supportability) and a

sharing policy with periodic access review.

- A full **lifecycle and retirement** process — including the block-then-delete discipline that makes

retirement safe and evidenced.

- **Five read-only Microsoft Graph PowerShell scripts** to baseline the estate, validate the

governance gates, produce a trendable scorecard, assemble approval evidence, and flag broadly-shared agents for review.

- The exact **admin-center enforcement steps** (Copilot Control System agent access and sharing

controls; Agent Registry block/reassign/publish), verified against Microsoft Learn on the build date.

How it connects to what you already own

The model **references, rather than rebuilds**, your existing Copilot controls: it triggers the Copilot data-governance / DLP guardrails and the SharePoint oversharing remediation where a data review demands them, and it consumes the deep agent inventory/export from its sibling product. It watches the admin center's **"Agents at risk"** signals (aggregated from Microsoft Entra, Purview, and Defender) and defines who acts on them — without reconfiguring those platforms.

Why run it yourself

The package is self-guided, click-by-click documentation a novice administrator can follow end to end, plus tested scripts and every governance artefact pre-named. You do not need a consultant to stand up agent governance: you need the operating model, the exact admin surfaces, and the discipline of a pilot-first rollout — all of which are here.

The outcome

Within two board cycles you move from an unknown, ungoverned agent estate to a measured one: every business agent owned, sensitive agents reviewed, org-wide shares approved, ownerless agents reassigned or retired, and a monthly scorecard that proves the model is working. The delta between the Step 1 baseline and the first steady-state scorecard is your business case — fewer ownerless agents, fewer unreviewed org-wide shares, and a defensible audit trail for every approval.

The governance process is identical for Commercial and US Government (GCC / GCC High / DoD) tenants; only portal URLs and agent feature availability differ, and both are documented inline. Agent admin surfaces evolve quickly — volatile specifics are flagged "verify against current Microsoft documentation."

Solution Overview

The problem this solves

Your users can now build their own AI agents. In the **Agent Builder** experience inside Microsoft 365 Copilot, a business user describes what they want in plain language, points the agent at some SharePoint sites, Teams chats, or public websites for knowledge, clicks **Create**, and then shares the result with a colleague, a team, or the entire organisation — all without writing code and, in most tenants, without anyone in IT ever seeing it.

That is a genuine productivity breakthrough. It is also a governance gap, because these business-process agents raise exactly the questions that no packaging tool, no DLP policy, and no label taxonomy answers:

- **Who owns this agent?** Not "who clicked Create" — who is accountable when it gives a wrong

answer to a customer, when its instructions drift, or when Security asks whether it still needs the data access it has.

- **Who said it could go into business use?** Copilot Studio agents can be routed through an

admin approval gate before publication. A declarative agent built in **Agent Builder** and shared directly with "Anyone in your organization" **can reach the whole tenant without any approval at all** — that is the specific gap this product closes.

- **Who is it shared with, and should it be?** An agent built over a Finance SharePoint site and shared org-wide is a data-exposure decision that was never reviewed.
- **When does it get retired?** The user who built it has left. The agent still runs, still answers, still holds its knowledge sources. Nobody decides to switch it off.

When these questions have no answer, the tenant accumulates **ownerless agents** (the creator left, nobody maintains them), **unreviewed agents** (no data or sharing review, quietly reaching more people than intended), and **shadow agents** (running with no registry entry and no accountable owner). Better tooling does not fix this. What fixes it is an **operating model**: named roles, a defined intake path, a decision forum with authority, published standards, a sharing policy with review, a lifecycle with real retirement, and metrics that prove it works.

This solution is that operating model. It is deliberately a *governance* product, scoped to the low-code, business-user-built **declarative agents** created in **Agent Builder** (and their Copilot Studio equivalents) that are used inside business processes.

What this solution is NOT

This package does **not** teach anyone how to *build* an agent, and it does **not** rebuild the sibling products it depends on. Specifically, it does **not** cover:

- **How to author a declarative agent** — instructions, knowledge sources, prompt design, capabilities. That is the maker's job in Agent Builder / Copilot Studio.
- **The technical inventory and sharing-posture export** — enumerating every agent, its sharing links, and the `copilotPackages` Graph diagnostic in depth. That is delivered by the sibling product *Copilot Agent Builder — Inventory, Sharing Governance & Export*, which owns the Graph endpoint diagnostic. This solution **consumes** that inventory as an input to the register; it does not reproduce the export tooling. (A lightweight governance-baseline discovery is included so you can start without the sibling, but the deep, repeatable export lives there.)
- **Copilot data governance with Microsoft Purview** — sensitivity labels, DLP for Microsoft 365 Copilot, auto-labelling, audit/eDiscovery of Copilot interactions. Delivered by the Copilot data governance / DLP guardrails solution. This model defines *when* a data review is required and *who* is accountable; it does not re-teach how to configure a DLP policy.
- **SharePoint / OneDrive oversharing remediation** — fixing who can reach what. Delivered by the Copilot oversharing / SharePoint-permissions solution. Agents inherit that access posture; this model references it as a prerequisite, not a deliverable.

What it **does** cover is the layer above all of that: **who may build a business agent, on what basis it is allowed into business use, who it may be shared with, who is accountable afterwards, and when it is retired** — plus exactly how each of those decisions is expressed and enforced in the Microsoft 365 admin center (the **Copilot Control System** and the **Agent Registry**) so the governance is enforced rather than aspirational.

The seven components of the operating model

The model has seven components. Each is defined in this package, and each produces a **named, durable artefact** the organisation keeps.

1. Ownership and RACI

Every business-process agent has **four named accountabilities**:

Role	Accountable for	Typically held by
Agent Owner	The agent itself — its instructions, knowledge sources, behaviour, runbook, and keeping it correct. Usually the maker who built it.	The business user or power user who created the agent.
Business Sponsor	The business need. Whether the agent should exist at all, the value it delivers, and its retirement.	A business function lead (Finance, HR, Sales, Clinical).
Data & Security Reviewer	The risk: what data the agent can reach, how widely it is shared, sensitivity/label exposure, and DLP posture.	Information Security / data governance.
Agent Admin	Enforcing decisions in the Microsoft 365 admin center — approve, publish, block, reassign, retire.	Copilot / M365 administrator holding the **AI Administrator** role.

The artefact is the **AGBGOV – Agent Register** — the authoritative list of every business-process agent, its four accountabilities, its review status, and its lifecycle state. Section 3 (Step 3) builds it, and `AGBGOV-Discovery.ps1` seeds it from your tenant.

2. Build eligibility and the intake / approval gate

Two decisions, made explicit:

• **Who may build agents at all.** Not every user needs to. Agent creation is governed centrally in

the Copilot Control System (agent access settings) and, for org-wide sharing, by an explicit policy.

• **The gate before an agent enters business use.** The customer is right that Copilot Studio's own

publish flow already routes *its* agents through an admin approval before they reach the org catalog. The gap is the **Agent Builder** declarative agent shared directly with **"Anyone in your organization"**, which bypasses that gate. This model closes the gap by (a) restricting org-wide sharing to an approved group and (b) requiring every agent used in a business process — however it was built — to pass the **AGBGOV – Agent Review Board** before it is treated as sanctioned.

The artefacts are the **AGBGOV – Agent Intake Request Form** and the **AGBGOV – Agent Intake Queue**.

3. The Agent Review Board (ARB)

A standing decision forum with **explicit decision rights**. The ARB can say *approved*, *approved with conditions*, *deferred pending evidence*, or *rejected*, and its decision is binding. It meets on a fixed cadence, has defined membership and quorum, and requires a defined **evidence pack** — including the agent's data-access and sharing posture — before an item is tabled.

The artefact is the **AGBGOV – Agent Decision Record** — one entry per decision, retained for audit.

4. Standards and policy

A published definition of what **"approved for business use"** means, so approval is a test rather than an opinion. Covers the data-and-sharing review criteria, the naming standard applied in the Agent Registry, the sharing-scope rules, and the supportability bar an agent must clear.

The artefact is the **AGBGOV – Agent Standards – Enterprise – v1.0** and the **naming convention** applied to agents.

5. Sharing policy and access review

The rules for **who an agent may be shared with**, the requirement for a data-access review **before broad sharing**, and a **periodic access review** of agents that are already shared widely. This is the component that turns "shared with Anyone in your organization" from an unreviewed default into a governed decision.

The artefact is the sharing-scope policy inside the standards document and the **AGBGOV – Shared-Agent Access Review** log, driven by `AGBGOV-Review-SharedAgents.ps1`.

6. Lifecycle and change control

An agent is registered, changed, reviewed on a cadence, and — critically — **retired** when it is stale, ownerless, or risky. The model classifies every change into a **risk tier** with a defined approval path, and it defines the retirement / decommission process end to end, with evidence.

The artefacts are the **AGBGOV – Agent Change Classification – Enterprise – v1.0** and the **AGBGOV – Agent Retirement Log**.

7. Metrics and evidence

The measures that prove the model works, and the evidence trail that proves it to an auditor. Coverage (owned, reviewed, ring-appropriate), flow (intake-to-decision time), quality (approval adherence, retirement completion), and risk (ownerless agents, unmanaged/shadow agents, overdue reviews, broadly-shared agents).

The artefact is the **AGBGOV – Agent Governance Scorecard**, produced by `AGBGOV-Report.ps1`.

How declarative agents are built, shared, approved, and managed (verified against Microsoft Learn)

Verified against Microsoft Learn on 2026-07-24. Agent admin surfaces evolve quickly; where a control is preview or licence-gated it is flagged. Verify against current Microsoft documentation before relying on volatile specifics.

- **Built:** A declarative agent is created in **Agent Builder** inside Microsoft 365 Copilot (or in

SharePoint, or as a "Copilot agent" in **Copilot Studio**) using natural language, with knowledge sources from SharePoint, Teams, Outlook, public websites, or Microsoft 365 Copilot connectors. Agent Builder requires a Microsoft 365 Copilot licence or a pay-as-you-go Copilot Studio tenant.

- **Shared (Agent Builder):** the maker chooses one of three sharing options — **Only you** (default,

private), **Specific users in your organization** (named users, security groups, security-enabled Microsoft 365 groups, or teams), or **Anyone in your organization** (an org-wide sharing link). Sharing gives direct

access; it does not deploy the agent across channels.

• **Governed sharing:** in the Microsoft 365 admin center an admin sets who may create org-wide

sharing links — **All users** (default), **Specific users or groups**, or **No users** (org-wide sharing disabled). Changes apply only to *new* sharing actions; already-shared agents keep their access until the owner updates them.

• **Approved / published:** to become broadly discoverable, an agent is **submitted to the org

catalog**; an admin** reviews the request **in the admin center and** publishes **it to the** Built by your org **section of the** Agent Store**, or** rejects** it. Copilot Studio and custom-engine agents publish through the same admin-approval path.

• **Managed:** the **Agent Registry** (Microsoft 365 admin center → **Agents → All agents →

Registry**) is the central inventory of every agent — Microsoft, external partner, published by your org, and shared by creator. From it an admin can** block/unblock**,** delete**,** reassign an ownerless agent's owner**,** pin**,** export to CSV **(30+ fields incl. owner, publisher, channel, status, instructions), and see** Agents without owners**,** Unmanaged agents**, and** Agents at risk** (risk signals aggregated from Microsoft Entra, Purview, and Defender).

• **Roles:** the least-privilege admin role for agent governance is **AI Administrator**; **Global

Reader **is view-only;** Global Administrator** is highly privileged and reserved for emergencies.

How this governance plugs into your existing Copilot controls (without re-teaching them)

Governance that lives only in a document is decoration. This model expresses each decision as a concrete control in the Copilot Control System and connects to the security stack you already own:

Governance decision	Expressed / enforced as
"Only approved people may share agents org-wide."	Copilot Control System **agent sharing** setting → **Specific users or groups** = the `AGBGOV – Agent Sharing – Org-Wide Approved` group.
"Only sanctioned people may build agents."	Copilot Control System **agent access** setting → **Specific users/groups** = the `AGBGOV – Agent Builders` group.
"This agent is approved for business use."	Admin **publishes** it to **Built by your org** in the Agent Store after ARB approval; the ARB reference is recorded in the register.
"This agent is unsafe / non-compliant."	Admin **blocks** it in the **Agent Registry**.
"This agent has no owner."	The register names one; the admin **reassigns** the ownerless agent in the Agent Registry.
"The change was authorised and by whom."	The `AGBGOV – Agent Decision Record` plus the Agent Registry CSV export, packaged by `AGBGOV-Export-ApprovalEvidence.ps1`.

The model also defines the **interface** to your existing Copilot data controls without re-specifying them:

• **Microsoft Purview (DLP for Copilot, sensitivity labels, audit)** — the intake form's data answers determine whether a **data review** is required before approval; the model defines the trigger and the accountable reviewer, and relies on the Copilot data-governance solution for the actual DLP/label configuration. Agents **honour** existing label-based access — an agent cannot surface content its user cannot open.

• **SharePoint / OneDrive oversharing** — an agent's data reach is only ever the *user's* existing access. Fixing oversharing is the oversharing-remediation solution's job; this model treats a known oversharing posture as a prerequisite and flags agents built over sensitive sites for review.

• **Microsoft Entra / Defender "Agents at risk" signals** — the admin center surfaces shadow agents, ownerless agents, excessive-permission agents, and sensitive-data-access risks aggregated from Entra, Purview, and Defender. The model defines the **cadence** for reviewing those signals and **who acts** on them — it does not reconfigure those platforms.

The standard deployment model

This solution follows the same shape as every other solution in the category: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

• **Assess** — baseline the agents already in the tenant and their sharing posture (`AGBGOV-Discovery.ps1`); identify ownerless and broadly-shared agents.

• **Configure** — stand up the roles, groups, register, standards, sharing policy, and forum, and set the Copilot Control System controls.

• **Pilot** — run the model over a limited scope (one business unit or one agent family) for two full ARB cycles.

• **Validate** — measure against the governance gates in Section 5 (`AGBGOV-Validation.ps1`).

• **Expand** — extend across the tenant and move to steady-state operation with the scorecard.

Limitations and honest scope boundaries

• **This is an operating model, not a purchase of maturity.** It works only if the organisation assigns the named roles and holds the forum. A model nobody attends is a document.

• **It requires an accountable executive sponsor** who will back the ARB when a popular agent is rejected or an org-wide share is refused. If you cannot identify that sponsor, secure it before Step 1.

• **The admin surfaces are moving fast.** Agent Builder, the Agent Registry, Agent 365, and the `copilotPackages` Graph API are new and partly in preview; some are licence-gated (the Graph Package Management API requires a **Microsoft Agent 365** licence). Every volatile control in this guide is flagged "verify against current Microsoft documentation."

• **It does not fix oversharing or configure Purview for you.** Those are prerequisites delivered by

sibling solutions; this model defines when they must be invoked, not how to configure them.

• **Metrics require the register to be maintained.** The scripts can measure what the tenant exposes;

they cannot measure business ownership that was never recorded.

A note for Government tenants (GCC / GCC High / DoD)

The **operating model itself** — the roles, decision rights, intake gate, sharing policy, lifecycle, and artefacts — is **identical** for Commercial and US Government tenants. What differs is the **portal URLs** and the **feature availability**:

Purpose	Commercial	US Government
Microsoft 365 admin center	`https://admin.microsoft.com`	`https://admin.microsoft.us`
Entra / Azure portal	`https://entra.microsoft.com` / `https://portal.azure.com`	`https://portal.azure.us`
Sign-in authority	`https://login.microsoftonline.com`	`https://login.microsoftonline.us`
Microsoft Graph endpoint	`https://graph.microsoft.com`	`https://graph.microsoft.us`

Two Government-specific facts, verified against Microsoft Learn on 2026-07-24 and **flagged as fast-moving — verify against current Microsoft documentation before you rely on them**:

• **Publishing agents to the organization *is* supported in GCC High (GCCH) and GCC Moderate (GCCM)**

— so the admin approval / org-catalog gate this model depends on exists in Government tenants.

• **The `copilotPackages` Graph Package Management API is NOT available in US Gov L4 (GCC High) or L5

(DoD) **— only in the global service. In Government tenants, therefore, seed the register from the** Agent Registry CSV export **in the admin center rather than from `AGBGOV-Discovery.ps1`'s Graph path; the script degrades to this automatically. Government agent feature availability (declarative agents with actions, etc.) is rolling out on its own schedule and is narrower than Commercial —** verify current GCC High agent feature availability** before promising a capability.

Because the *governance process* is identical and only the portals/feature-availability differ, this is a **single deliverable** with the Government path documented inline (there is no separate Government guide). Every script accepts an `-Environment` switch (`Global` / `USGov` / `USGovDoD`) so it targets the correct sovereign endpoint.

Requirements

Confirm every requirement below **before** you begin Section 3. Each entry follows the same shape: **Description** (what it is), **Why required** (what breaks without it), **How to verify** (exactly where to look), and **How to implement if missing**. Category-wide Copilot prerequisites (tenant, Copilot licensing, Entra identity/MFA, network/client readiness, baseline data governance) live once in

`global-requirements-copilot.md` — confirm those first; the requirements here are specific to governing **declarative business-process agents**.

> ***Roles named below are exact.*** *Where a Microsoft role or Graph scope is required, it is written verbatim so you can search for it. Reserve* ***Global Administrator*** *for emergencies; every routine task in this model has a least-privilege role.*

R1. Accountable executive sponsor

Description. A single named senior leader who owns the outcome of the agent governance programme and will back the Agent Review Board's decisions.

Why required. Decision rights are only real if someone senior will support the ARB when a director's favourite agent is rejected, or when a popular org-wide share is refused. Without a sponsor, the model becomes advisory and is ignored the first time it says "no".

How to verify. A named individual has agreed in writing to sponsor the programme and to receive the monthly `AGBGOV – Agent Governance Scorecard`.

How to implement if missing. Present the Section 1 problem statement and the Step 1 baseline (ownerless and broadly-shared agent counts) to your IT or data-governance leadership and secure a named sponsor before Step 1. Do not proceed without one.

R2. Microsoft 365 Copilot licensing (and awareness of the agent billing models)

Description. Microsoft 365 Copilot licences for the users who build and use agents, or a pay-as-you-go Copilot Studio subscription for metered agent consumption.

Why required. Agent Builder — the tool that creates the declarative agents this model governs — is available only to users with a Microsoft 365 Copilot licence **or** in tenants with pay-as-you-go enabled for Copilot Studio. Agents that reach shared tenant data (SharePoint, Graph connectors) are billed on metered consumption and are **off by default** in Copilot Chat. You must know which model you are in before you can govern who builds what.

How to verify. Microsoft 365 admin center → **Billing** → **Licenses** → **Subscriptions**; confirm Microsoft 365 Copilot seats and/or a Copilot Studio pay-as-you-go / capacity plan.

How to implement if missing. Assign Microsoft 365 Copilot licences (via group where possible) to the pilot cohort, or enable pay-as-you-go for Copilot Studio. Confirm current base-plan prerequisites in your tenant.

R3. The AI Administrator role (least-privilege agent governance)

Description. At least one administrator holding the **AI Administrator** Entra role to configure agent settings, manage the Agent Registry, and approve/publish/block agents.

Why required. Every enforcement action in this model — setting agent access and sharing controls, approving an org-catalog submission, blocking an unsafe agent, reassigning an ownerless agent, and

calling the `copilotPackages` Graph API — is available to **AI Administrator**. Using it instead of Global Administrator is the least-privilege posture Microsoft recommends and this model requires.

How to verify. Entra admin center → **Roles & admins** → **Roles & admins** → search **AI Administrator** → **Assignments**; confirm at least one member. Also confirm in Microsoft 365 admin center → **Users** → **Active users** → *your account* → **Roles**.

How to implement if missing. Assign the **AI Administrator** role to the person(s) who will hold the **Agent Admin** accountability. Assign **Global Reader** to auditors who need view-only access. Do not assign Global Administrator for routine agent work.

R4. Entra group-management rights

Description. Rights to create and own Microsoft Entra security groups (Groups Administrator, or a delegated group owner with creation rights).

Why required. The model's accountabilities and its two central controls — *who may build agents* and *who may share org-wide* — are bound to Entra security groups (`AGBGOV – Agent Builders`, `AGBGOV – Agent Sharing – Org-Wide Approved`, and the four accountability groups). Without group-management rights you cannot create the objects the controls point at.

How to verify. Entra admin center → **Identity** → **Groups** → **All groups** → **+ New group**; if you can open and complete the form, you have the right.

How to implement if missing. Have a Global Administrator assign **Groups Administrator**, or make your operator an owner of the relevant groups with tenant group-creation enabled.

R5. A system of record for the governance artefacts

Description. A place to hold the register, decision records, standards, and logs — typically SharePoint Online lists, or an ITSM CI class if you have one.

Why required. The governance *system of record* is distinct from the technical inventory. The Agent Registry in the admin center tells you what exists in the tenant; the `AGBGOV – Agent Register` tells you who is accountable, what was decided, and when it is next reviewed. One cannot substitute for the other.

How to verify. You can create a SharePoint list on your IT team's site (or a CI class in your ITSM tool) and set item-level permissions and version history.

How to implement if missing. Provision a SharePoint Online site for IT governance, or identify the ITSM class you will use. Step 3 builds the lists.

R6. Known Copilot data-governance and oversharing posture (prerequisite solutions)

Description. A known state of (a) SharePoint/OneDrive sharing and oversharing, and (b) Copilot data governance — sensitivity labels and DLP for Microsoft 365 Copilot.

Why required. An agent's data reach is exactly its user's existing access; a Finance agent shared org-wide over an overshared site is an exposure this model must *flag for review* but cannot *fix* — fixing it is the oversharing-remediation solution's job, and constraining what Copilot may process is the Copilot data-governance / DLP solution's job. This model defines *when* a data review is required and *who* owns it; it depends on those two solutions for the actual controls.

How to verify. Confirm whether the oversharing-remediation and Copilot data-governance/DLP programmes have run, or are in place, for the scope you intend to enable agents in.

How to implement if missing. Sequence those solutions ahead of, or alongside, broad agent enablement. You can still run the governance model in pilot scope, but record the oversharing/DLP posture as a known risk in each affected agent's register entry.

R7. Change-management interface (CAB), if one exists

Description. Knowledge of whether your organisation runs a formal Change Advisory Board (CAB) and, if so, its cadence and submission deadlines.

Why required. Higher-risk agent changes (an agent handling regulated data, or one being shared tenant-wide) should be represented in enterprise change control without every minor instruction tweak consuming a CAB slot. The model defines the CAB interface in Step 8; you need to know whether a CAB exists to complete it.

How to verify. Ask your service-management function whether a CAB operates and obtain its cadence.

How to implement if missing. If no CAB exists, the ARB holds change authority for all tiers — record that explicitly so an auditor is not left inferring it.

R8. Nominated role-holders

Description. Named individuals (or a documented way to nominate them per agent) for the four accountabilities: **Agent Owner**, **Business Sponsor**, **Data & Security Reviewer**, **Agent Admin**.

Why required. The RACI in Step 2 is only enforceable if the groups behind it have members. An empty `AGBGOV – Agent Data-Security Reviewers` group means no agent handling sensitive data can ever be approved.

How to verify. You have at least one nominee for Business Sponsor, Data & Security Reviewer, and Agent Admin, and a rule for who becomes Agent Owner (default: the maker who built the agent).

How to implement if missing. Work with Information Security and the sponsoring business functions to nominate role-holders before Step 4.

R9. Microsoft Graph PowerShell (for the governance scripts)

Description. The **Microsoft Graph PowerShell SDK** installed on the operator workstation, able to sign in to the correct cloud (`Global` / `USGov` / `USGovDoD`).

Why required. The five `AGBGOV-*.ps1` scripts use Microsoft Graph PowerShell (read-only by default) to baseline agents, validate governance gates, produce the scorecard, package approval evidence, and flag

broadly-shared agents.

How to verify. In PowerShell run `Get-Module Microsoft.Graph -ListAvailable`. Confirm the SDK is present and the account can `Connect-MgGraph`.

How to implement if missing. Install with `Install-Module Microsoft.Graph -Scope CurrentUser -Force`. For the `copilotPackages` discovery path, the signed-in account needs the delegated scope `CopilotPackages.Read.All` and a **Microsoft Agent 365** licence; where that is unavailable (including all Government tenants), the discovery script degrades to the Agent Registry CSV export — see Section 3, Step 1. *This API is preview/licence-gated — verify current availability in your tenant.*

R10. Understanding of the agent build surfaces in scope

Description. Clarity on which build surfaces this model governs: **Agent Builder** declarative agents (primary), **SharePoint** agents, and **Copilot Studio** "Copilot agents" (declarative). Full custom-engine agents and pro-code agents built with the Microsoft 365 Agents Toolkit are **out of scope** here (they follow the code app-lifecycle and already require admin publication).

Why required. Scoping the model to business-user declarative agents keeps it proportionate. The gap it closes is specifically the Agent Builder agent that can be shared org-wide with no approval; custom-engine agents already require admin approval to publish.

How to verify. Confirm with your makers which surfaces are in use. `AGBGOV-Discovery.ps1` reports each agent's **platform** (`Microsoft 365 Copilot Agent Builder` vs `Copilot Studio`) and **elementType** (`DeclarativeAgent` vs `CustomEngineAgent`) so you can see the mix.

How to implement if missing. If custom-engine agents are prevalent, run this model for the declarative agents and handle custom-engine agents through your existing application-change process; the RACI and ARB still apply.

Global Requirements — Copilot (category-level)

These requirements apply to every solution in the Copilot category. They are published once at the category level so individual listings stay small. Confirm these before deploying any Microsoft 365 Copilot solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Copilot inherits the tenant's existing identity, permission, and data-governance posture. It creates no new access — but it surfaces everything a user already has. Treat data readiness (oversharing, labeling) as a prerequisite, not an afterthought, and pilot before broad enablement. Agents extend this: they act within the building or invoking user's existing access, so the same posture applies.*

G1. Microsoft 365 tenant with supported workloads

Description: A Microsoft 365 tenant with Exchange Online, SharePoint Online, OneDrive, and Teams in use. Why: Copilot reasons over Microsoft Graph content across these workloads; they must be provisioned and healthy. Verify: Microsoft 365 admin center shows the services active; users have mailboxes and OneDrive provisioned. Implement: Provision/enable the M365 workloads and confirm users are licensed for them before adding Copilot.

G2. Microsoft 365 Copilot licensing

Description: Microsoft 365 Copilot licenses for the target users (a prerequisite base M365 plan applies). Why: Copilot is a per-user add-on; a user cannot use Copilot without an assigned license on a qualifying base plan. Verify: Admin center -> `Billing -> Licenses`; confirm Copilot licenses and free seats. *Verify current base-plan prerequisites in your tenant.* Implement: Purchase/assign Microsoft 365 Copilot licenses to the pilot cohort; assign via group where possible.

G3. Microsoft Entra ID identity and MFA

Description: Cloud identities in Entra ID with MFA and healthy sign-in. Why: Copilot honors identity and Conditional Access; a weak or broken identity plane undermines both access and security. Verify: Users sign in at `https://entra.microsoft.com` resources; MFA is enforced for the cohort. Implement: Ensure licensed, MFA-protected Entra accounts (see the Entra category solutions for identity hardening).

G4. Data readiness — permissions & oversharing controls

Description: A known state of SharePoint/OneDrive permissions and sharing (no unmanaged broad access). Why: Copilot — and any agent built over it — can surface any content the user has access to; oversharing becomes AI data exposure. Verify: Review sharing reports / SharePoint Advanced Management signals; run the solution's discovery script. *SAM/Restricted Content Discovery availability varies — verify in your tenant.* Implement: Remediate broad-access sites and links before broad Copilot/agent enablement (see the oversharing-remediation solution).

G5. Information protection baseline (sensitivity labels)

Description: A published sensitivity-label taxonomy (even minimal) in Microsoft Purview. Why: Labels are how you keep sensitive content out of AI processing where required and audit what Copilot uses. Verify: Purview compliance portal -> Information Protection shows published labels. Implement: Publish a baseline label set (see the Purview information-protection solutions) before enforcing AI guardrails.

G6. Administrative roles

Description: The least-privilege admin roles for the task — for agent governance, the **AI Administrator** role (with **Global Reader** for view-only auditors); reserve Global Administrator for emergencies. Why: Enabling Copilot, remediating SharePoint, configuring Purview DLP/labels, and governing agents each require specific roles. Verify: Entra -> `Roles and administrators`; confirm the required roles. Implement: Assign least-privilege roles for the task; reserve Global Administrator for tenant-wide changes.

G7. Network and client readiness

Description: Supported Microsoft 365 Apps clients and network access to Microsoft 365/Copilot endpoints. Why: Copilot experiences in Word/Excel/Outlook/Teams — and Agent Builder — require current clients and connectivity. Verify: Clients on a supported channel/build; Microsoft 365 endpoints reachable. *Verify current client requirements.* Implement: Update the M365 Apps channel/build and allow the required endpoints.

G8. Governance & responsible-use policy

Description: An organizational position on acceptable AI use, data handling, and auditability. Why: Copilot and agents generate and surface content; the organization must define what is acceptable and keep an audit trail. Verify: A documented AI-use/acceptable-use policy exists; Copilot interactions are auditable (Purview audit). Implement: Adopt an AI-use policy; enable audit/eDiscovery for Copilot. *Governance and acceptable-use determinations are the customer's compliance/legal decision — this guide covers the technical controls only.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

Implementation

This section stands up the operating model end to end. Work through the steps **in order** — later steps depend on objects created by earlier ones. Every object you create is given an **exact name**; use that name verbatim, because Section 5 (Testing), Section 6 (Troubleshooting), and all five PowerShell scripts refer to these names directly.

Naming convention used throughout: `AGBGOV – <Purpose> – <Scope>`, using an en dash (–) surrounded by single spaces. Keep it consistent; the validation script matches on it.

> ***Government tenants (GCC / GCC High / DoD):*** *substitute* `https://admin.microsoft.us` *for* `https://admin.microsoft.com` *and* `https://portal.azure.us` *for* `https://entra.microsoft.com` *in every navigation path below. Blade, tab, and field names are the same. Two behaviours differ and are called out in the relevant steps: the* `copilotPackages` *Graph API is* ***not*** *available in Government (use the CSV export), and some agent features roll out later — **verify current GCC High agent feature availability**.*

Total elapsed time: expect 3–5 working days of effort for Steps 1–11, then two full board cycles (typically 4 weeks at a fortnightly cadence) for the pilot in Step 12.

Step 1 — Baseline the current agent estate and its sharing posture

You cannot govern what you have not counted. Produce the "before" picture first; the ownerless and broadly-shared counts are also what will hold the executive sponsor's attention.

1. On your admin workstation, open **PowerShell** as your normal (non-elevated) user.
2. Install the Graph SDK if it is not present:

```
Install-Module Microsoft.Graph -Scope CurrentUser -Force
```

1. Change to the folder containing this package's scripts.
2. Run the discovery script, exporting to CSV:

```
powershell -ExecutionPolicy Bypass -File .\AGBGOV-Discovery.ps1 -CsvPath .\AgentEstate-Baseline.csv
```

For a Government tenant add `-Environment USGov` (or `-Environment USGovDoD`).

1. At the `Connect-MgGraph` prompt, sign in with an account holding **AI Administrator** (R3) and

Accept the consent prompt for the read-only scopes.

1. **If the Graph path is unavailable** (you see *"copilotPackages API not available / not

licensed"* — this is expected in Government tenants and in tenants without a Microsoft Agent 365 licence), the script tells you to export the inventory manually instead. Do that now:

- Navigate to `https://admin.microsoft.com` → **Agents** → **All agents** → **Registry**.
- Click **Export** to download the agent list as CSV (it includes name, status, channel, date

created, last modified, publisher, publisher type, version, owner, description, platform, and instructions — 30+ fields).

- Re-run the script pointing at that file:

```
powershell -ExecutionPolicy Bypass -File .\AGBGOV-Discovery.ps1 -RegistryCsv .\ExportedAgents.csv -CsvPath .\AgentEstate-Baseline.csv
```

1. Open `AgentEstate-Baseline.csv` in Excel. Record these five numbers — they are your baseline

scorecard, and Step 12 will measure against them:

Metric	Where it comes from
Total declarative agents	Rows where `ElementType` = `DeclarativeAgent`
Agents **shared org-wide** (`AvailableTo` = `all`)	The broad-sharing exposure
Agents with **no owner** recorded	`Owner` is blank
Agents built in **Agent Builder** (the gap surface)	`Platform` = `Microsoft 365 Copilot Agent Builder`
Unmanaged / shadow agents	Flagged by the Agent Registry as unmanaged or lacking a registry entry

1. In the Microsoft 365 admin center, open **Agents** → **All agents** and note the **Agents without

owners**,** Unmanaged agents**, and** Agents at risk** tiles. Record those three numbers — they are usually the most uncomfortable and most useful figures in the baseline.

1. Save all files to your system of record (R5) in a folder named `AGBGOV – Baseline – <YYYY-MM>`.

Do not skip this; Step 12 compares against it.

Step 2 — Define the four accountabilities (the RACI)

1. Create a document in your system of record named exactly

`AGBGOV – Agent Governance RACI – Enterprise – v1.0`.

1. Populate it with the following matrix. This is the model's authority definition; adapt names, not

structure.

Activity	Agent Owner	Business Sponsor	Data & Security Reviewer	Agent Admin	ARB
Raise intake request for a new business agent	**A/R**	C	I	I	I
Provide business justification & value	C	**A/R**	I	I	C
Declare knowledge sources & data reached	**A/R**	I	C	I	C
Perform data-access & sharing review	C	I	**A/R**	I	C
Approve for business use	C	C	C	I	**A/R**
Publish the agent to "Built by your org"	I	I	I	**A/R**	I
Set / restrict org-wide sharing scope	C	I	C	**A/R**	I
Classify a change's risk tier	**A/R**	I	C	I	C
Block an unsafe / non-compliant agent	I	I	C	**A/R**	I
Own the agent runbook & keep it correct	**A/R**	I	I	I	I
Reassign an ownerless agent	I	C	I	**A/R**	I
Approve retirement / decommission	C	**A/R**	C	I	**A**
Sign off the periodic review	**A/R**	C	**C**	I	**A**

A = Accountable (one only), R = Responsible, C = Consulted, I = Informed.

1. Immediately below the matrix, record **the four escalation rules** verbatim:

• An agent used in a business process with no named **Agent Owner** is blocked, not tolerated.

• An agent handling `Confidential` or `Restricted` data with no completed **Data & Security Reviewer** sign-off cannot reach `Approved – business use`.

• Sharing an agent with **"Anyone in your organization"** requires an ARB approval *and* a completed data-access review; an unreviewed org-wide share is a finding.

• **Blocking an agent never requires a meeting.** Any **Agent Admin** may block any agent immediately on a credible safety or compliance concern; it is always a safe, reversible action.

1. Have the executive sponsor from R1 approve the document, and record the approval date in it.

Step 3 — Create the Agent Register, Decision Record, and Retirement Log

The register is the governance system of record. The Agent Registry in the admin center is the *technical* record of what exists; the `AGBGOV – Agent Register` is the *governance* record of who is accountable, what was decided, and when it is next reviewed.

1. Open your system of record from R5. These instructions use SharePoint Online; adapt the field types if you are using an ITSM CI class.

1. Navigate to your IT team's SharePoint site → **New** → **List** → **Blank list**.

2. In **Name**, enter exactly `AGBGOV – Agent Register`. In **Description**, enter `Authoritative record of every business-process agent, its accountabilities, approval status, sharing scope, and lifecycle state.` Leave **Show in site navigation** ticked. Click **Create**.

1. Add the following columns. For each, click **+ Add column**, choose the type shown, enter the name exactly as written, then click **Save**.

Column name	Type	Notes
`Agent Name`	Single line of text	Rename the default `Title` column to this. Match the Agent Registry display name.
`Agent Id`	Single line of text	The package/agent id from the Agent Registry; the join key to the scripts.
`Build Surface`	Choice	`Agent Builder`, `SharePoint`, `Copilot Studio`.
`Agent Owner`	Person	The maker accountable for the agent. Required.

Column name	Type	Notes
`Business Sponsor`	Person	Business accountability for the need. Required.
`Data & Security Reviewer`	Person	Who signed the data/sharing review.
`Agent Admin`	Person	Who enforces in the admin center.
`Knowledge Sources`	Multiple lines of text	The SharePoint sites / sources the agent reads.
`Data Classification`	Choice	`Public`, `Internal`, `Confidential`, `Restricted`.
`Sharing Scope`	Choice	`Only owner`, `Specific users/groups`, `Anyone in org`.
`Business Justification`	Multiple lines of text	Copied from the intake request.
`Lifecycle State`	Choice	`Requested`, `In review`, `Approved – pilot`, `Approved – business use`, `Deprecated`, `Retired`.
`ARB Decision Reference`	Single line of text	Format `AGBARB-<YYYY>-<NNN>`.
`Approval Date`	Date and time	Date only.
`Next Review Date`	Date and time	Approval date + review interval (Step 6).
`Risk Tier`	Choice	`Tier 1 – Low`, `Tier 2 – Moderate`, `Tier 3 – High`, `Tier 4 – Critical`.
`Runbook Link`	Hyperlink	Points at the agent runbook.
`Published To Store`	Choice	`Not published`, `Built by your org`.

1. Enable version history: **gear icon** → **List settings** → **Versioning settings** → set **Create a version each time you edit an item in this list **to** Yes → OK**.

1. Set permissions: **gear icon → List settings → Permissions for this list → Stop Inheriting Permissions → OK**. **Grant the register custodian** Edit**; grant all Agent Owners** Read**. Approval fields must not be editable by requesters.

1. Seed the register from your Step 1 baseline. For each declarative agent create a register item populating `Agent Name`, `Agent Id`, `Build Surface`, and `Sharing Scope`. Leave the accountability and approval fields blank — the gaps are the point, and Step 11 fills them.

1. Create a second list named exactly `AGBGOV – Agent Decision Record` with columns: `Decision Reference (single line of text, format AGBARB-<YYYY>-<NNN>), Agent Name (single line of text), Decision Date (date), Decision (choice: Approved, Approved with

conditions, Deferred – evidence required, Rejected), Conditions (multiple lines of text), Members Present (multiple lines of text), Data Review Outcome (multiple lines of text), Evidence Pack Link (hyperlink), Decision Rationale` (multiple lines of text). Enable version history as in step 5.

1. Create a third list named exactly AGBGOV – Agent Retirement Log with columns: Agent Name (single line of text), Agent Id (single line of text), Retirement Reason (choice: Owner left / ownerless, Superseded, No longer used, Failed review, Security/compliance), Decision Reference (single line of text), Blocked Date (date), Deleted Date (date), Retired By (person), Evidence Link (hyperlink). Enable version history.

Step 4 — Create the governance security groups in Entra ID

These groups make the RACI enforceable, and they are what the two Copilot Control System controls in Step 5 are bound to.

1. Navigate to https://entra.microsoft.com and sign in with an account holding the rights from R4.
2. In the left navigation, expand **Identity**, then click **Groups** → **All groups**.
3. Click **+ New group**.
4. Complete the **New Group** form for the first group:

• **Group type:** Security

• **Group name:** AGBGOV – Agent Builders

• **Group description:** Users sanctioned to build declarative agents (Agent Builder / Copilot Studio). Bound to the Copilot agent access control.

• **Microsoft Entra roles can be assigned to the group:** No

• **Membership type:** Assigned

• **Owners:** add the register custodian.

• **Members:** add the sanctioned makers nominated in R8.

1. Click **Create**.
2. Repeat steps 3–5 exactly to create each of the following. **Use** Assigned **for all** — these are deliberate governance decisions, and a dynamic rule would let an HR attribute change silently grant or remove build/share rights.

Group name	Description
AGBGOV – Agent Sharing – Org-Wide Approved	Users permitted to create org-wide ("Anyone in your organization") agent sharing links. Bound to the Copilot agent sharing control.

Group name	Description
`AGBGOV – Agent Owners`	`Makers accountable for individual agents, their instructions, knowledge sources, and runbooks.`
`AGBGOV – Agent Business Sponsors`	`Business leads accountable for the need and value of agents.`
`AGBGOV – Agent Data-Security Reviewers`	`Information Security / data governance staff who review agent data access and sharing.`
`AGBGOV – Agent Admins`	`Administrators (AI Administrator role) who approve, publish, block, reassign, and retire agents.`

1. Verify: still in **Identity** → **Groups** → **All groups**, type `AGBGOV` in the search box. Exactly six groups must be listed. Record each group's **Object Id** (open the group → **Overview** → copy **Object Id**) into the RACI document from Step 2.

Step 5 — Set the enforcement controls in the Copilot Control System

This is the step that converts "only sanctioned people may build, and only approved people may share org-wide" from a sentence into an enforced control. *Agent admin surfaces evolve quickly — the exact blade labels below were verified against Microsoft Learn on 2026-07-24; if a toggle has moved, find its current location and apply the same intent. Verify against current Microsoft documentation.*

5a — Restrict who can build / access agents

1. Sign in to the **Microsoft 365 admin center** (`https://admin.microsoft.com`) with **AI Administrator** (R3).
1. Open the **Copilot Control System**, then navigate to **Copilot → Settings → Data access → Agents**.
1. Under who can access agents, select **Specific users/groups** and choose `AGBGOV – Agent Builders`. (Choose **All users** only if your policy is open build with post-hoc governance; choose **No users** to pause agent access entirely.)
1. Under which types of apps and agents are available, choose the publishers your policy allows — agents created by **Microsoft**, **external publishers**, and **your organization** — and **uncheck** any you do not sanction. Review the third-party data-handling warning before allowing external publishers.
1. Click **Save**.

5b — Restrict who can share agents org-wide

1. Still in the Microsoft 365 admin center, open the **agent sharing** control (in the Copilot Control System agent settings; *verify the current blade path*).

1. Set org-wide sharing to **Specific users or groups** and choose

`AGBGOV – Agent Sharing – Org-Wide Approved`. This is the control that closes the core gap: an Agent Builder agent can no longer be shared with **"Anyone in your organization"** by an unapproved user.

• Choose **No users** if your policy is that *no* agent may be shared org-wide without going

through the org-catalog publish flow (the strictest posture).

• **All users** is the default and the posture this model exists to replace.

1. Click **Save**. Note the documented behaviour: **the change applies only to new sharing actions**

— agents already shared org-wide keep that access until their owner updates them. Step 9 handles the existing broadly-shared agents.

5c — Confirm the org-catalog approval path is on

1. Confirm that submissions to the org catalog route to admin review: Microsoft 365 admin center →

Agents → **All agents** → **Registry**, and confirm you (as Agent Admin) receive and can act on **agent requests** (review → **publish** to *Built by your org*, or **reject**). This is the gate every business agent passes in Step 8.

1. *Government note:* publishing agents to the organization **is** supported in GCC High and GCC

Moderate — confirm the request queue is visible in your Government admin center.

Step 6 — Publish the Agent Standards document

This is the document that makes "approved for business use" a test rather than an opinion.

1. Create a document in your system of record named exactly `AGBGOV – Agent Standards – Enterprise –

v1.0`.

1. Include the following six sections. Each must be written so a reviewer can answer *yes* or *no* —

avoid "should be considered".

Section 1 — Definition of approved for business use. An agent is `Approved – business use` only when all of the following are true: a named Agent Owner and Business Sponsor exist in the register; the data-access & sharing review is complete and signed; the sharing scope is set no broader than the review permits; a runbook exists and is linked; the agent's name conforms to the naming standard; and an `AGBGOV – Agent Decision Record` exists with a decision of `Approved` or `Approved with conditions` (all conditions met).

Section 2 — Data-access & sharing review criteria. The Data & Security Reviewer must record a determination against each of: the data classification the agent handles; the specific knowledge sources (SharePoint sites/files, connectors, websites) it reads; whether any source is known to be overshared (cross-reference the oversharing-remediation posture, R6); whether a **DLP for Microsoft 365 Copilot** policy or sensitivity-label boundary applies (cross-reference the Copilot data-governance solution — do not re-specify it here); the requested sharing scope and whether it is justified; and whether the agent should be published to the org catalog or kept to specific users.

Section 3 — Sharing standard. Default sharing is **Only owner** during build and **Specific users/groups** for team use. **"Anyone in your organization"** is permitted only with ARB approval and a completed data review, and only for agents whose knowledge sources are all `Public` or `Internal`. Publishing to **Built by your org** is the sanctioned route for broad availability because it passes admin review.

Section 4 — Naming standard. Every business-process agent must be named:

```
AGBGOV – <Function> – <Purpose> – <Audience>
```

For example: `AGBGOV – Finance – Expense Policy Q&A – Finance Team`, `AGBGOV – HR – Benefits Assistant – All Staff`, `AGBGOV – Sales – Proposal Helper – Sales EMEA`. Use the en dash with single spaces. `AGBGOV-Validation.ps1` tests conformance to exactly this pattern for agents in scope.

Section 5 — Supportability bar. Before business-use approval an agent must have: a documented purpose and the instructions it runs on; a named Agent Owner who will keep it correct; a runbook covering how to update it, how to reassign it if the owner leaves, and how to retire it; and a statement of the data it reaches.

Section 6 — Review interval. Set `Next Review Date` by risk tier: `Tier 4 – Critical` every 6 months; `Tier 3 – High` every 12 months; `Tier 1 – Low` and `Tier 2 – Moderate` every 12 months (agents drift faster than packaged apps — do not exceed 12 months for any tier). A review that lapses moves the agent to `Deprecated` until it is re-reviewed.

1. Have the executive sponsor and the Head of Information Security approve the document. Record both

approval dates.

1. Publish it where every maker can read it **before** they build. A standards document nobody can

find generates the same rework as no standards document.

Step 7 — Stand up the intake workflow

1. Create the intake form. Using Microsoft Forms: navigate to `https://forms.office.com` → **New

Form → **name it exactly** `AGBGOV – Agent Intake Request Form`**. Add these questions, all** Required** unless stated:

#	Question	Type
1	Proposed agent name (per the naming standard)	Text
2	Named Agent Owner (the maker) and Business Sponsor	Text
3	What business process will this agent support, and what happens if we do not build it?	Long text

#	Question	Type
4	Which knowledge sources will it read? (SharePoint sites/files, Teams, connectors, websites)	Long text
5	Highest data classification among those sources	Choice: Public / Internal / Confidential / Restricted
6	Who needs to use it?	Choice: Only me / Specific team or group / Anyone in the organisation
7	Build surface	Choice: Agent Builder / SharePoint / Copilot Studio
8	Does any knowledge source contain personal, financial, health, or regulated data?	Choice: Yes / No / Unknown
9	Required-by date and the driver for it	Text
10	Has an existing approved agent been evaluated for this need? Which, and why insufficient?	Long text

1. Route the responses into a SharePoint list named `AGBGOV – Agent Intake Queue` (via the form's

Responses → ■ → **Create a Flow**), or export responses to Excel weekly if you have no Power Automate licence. The discipline matters more than the mechanism.

1. Publish the **intake SLA**: the register custodian triages every request within **3 business

days** and returns it, tables it for the next ARB, or fast-tracks it under the Tier 1 pre-approval rule (Step 8).

1. Publish the **return criteria** explicitly. A request is returned to the requester — not queued —

when it lacks a named Agent Owner and Business Sponsor, a description of knowledge sources, or a data classification. Publish these so returns are not perceived as arbitrary.

Step 8 — Stand up the Agent Review Board and change classification

1. Create a recurring Outlook meeting named exactly

`AGBGOV – Agent Review Board – Enterprise – Fortnightly`.

1. Set the cadence and duration:

• **Cadence:** every two weeks. Weekly is unsustainable; monthly makes IT the bottleneck and

drives users to share agents around the gate.

• **Duration:** 45 minutes. If you consistently overrun, the intake gate in Step 7 is too loose.

1. Define **membership**. Invite as **required**: the register custodian (chair), one member of

AGBGOV – Agent Data-Security Reviewers, and one AGBGOV – Agent Admins member. Invite as **optional**: the requesting Agent Owner and Business Sponsor for their item only.

1. Define **quorum** and record it in the standards document: the chair plus one Data & Security

Reviewer. **Without a Data & Security Reviewer present, no agent handling** Confidential **or** Restricted **data and no request to share org-wide may be approved** — such items are deferred, not waved through.

1. Define the four permitted **decisions**. The board selects exactly one per item:

• Approved — proceeds to Approved – pilot, then business use.

• Approved with conditions — proceeds, with each condition recorded with a named owner and due

date (e.g. "narrow sharing to the Finance group", "remove the HR site as a knowledge source").

• Deferred – evidence required — the specific missing evidence is named, with who supplies it

and by when (commonly a completed data-access review).

• Rejected — with a written rationale that states what would change the answer.

1. Define the **evidence pack** required before an item may be tabled. The chair refuses to table

items lacking any of: the completed intake request; the data-access & sharing review determination against all criteria in Standards Section 2; the proposed sharing scope; a proposed risk tier; and a named Agent Owner who has confirmed they will keep the runbook.

1. Define the **Tier 1 pre-approval (standard change) rule** so the board is not consumed by routine

work: a minor instruction or wording change to an already-approved agent, with **no** change to knowledge sources, data classification, or sharing scope, is **pre-approved** and does not require an ARB slot. It is recorded retrospectively in the AGBGOV – Agent Decision Record with the reference AGBARB-<YYYY>-STD-<NNN>.

1. Create the change classification document named exactly `AGBGOV – Agent Change Classification –

Enterprise – v1.0`, with the four risk tiers:

Risk tier	Definition	Approval required	Sharing/publish policy	Rollback
Tier 1 – Low	Wording/instruction tweak; no change to sources, data, or sharing.	Pre-approved (standard). Owner confirms classification.	No change to sharing.	Owner reverts instructions.
Tier 2 – Moderate	New agent over Public/Internal sources; or a new knowledge source at the same classification.	ARB approval.	Specific users/groups; org-wide only with review.	Owner unshares / reverts.

Risk tier	Definition	Approval required	Sharing/publish policy	Rollback
`Tier 3 – High`	Agent over `Confidential` data; or any request to share **Anyone in your organization**; or publish to **Built by your org**.	ARB approval **and** Data & Security Reviewer sign-off.	Org-wide/publish only after review; CAB notified.	Agent Admin **blocks** in the Agent Registry.
`Tier 4 – Critical`	Agent over `Restricted`/regulated data; or a business-critical or clinical/regulated process.	ARB approval, Data & Security sign-off, **and** CAB approval.	Publish only via org catalog with a written go/no-go.	Agent Admin **blocks** immediately; Sponsor + CAB notified.

1. Define the **CAB interface** (R7): if a CAB exists, Tier 1–2 are standard/pre-approved and consume

no CAB time, Tier 3 is **notified** to CAB, and Tier 4 is **submitted** to CAB with the ARB Decision Reference as the technical approval evidence. If no CAB exists, the ARB holds change authority for all tiers — record that explicitly.

1. State the **blocking principle** prominently: *blocking an agent never requires approval.* Any

Agent Admin may block any agent at any time on a credible safety or compliance concern; blocking is reversible (unblock) and is the cheapest possible containment. Requiring approval to block converts a two-minute containment into a data-exposure window.

1. Record every decision in the `AGBGOV – Agent Decision Record` within **1 business day** of the

meeting. Decisions not recorded within that window are treated as not made.

Step 9 — Set the sharing policy and stand up the shared-agent access review

This is the component that turns broad sharing from an unreviewed default into a governed, reviewed decision — and cleans up the existing broadly-shared agents that Step 5b's control did not retroactively change.

1. Publish the sharing-scope rules from Standards Section 3 to all makers.
2. Run the shared-agent review to find every agent currently shared broadly:

```
powershell -ExecutionPolicy Bypass -File .\AGBGOV-Review-SharedAgents.ps1 -CsvPath .\SharedAgents-Revie
```

For a Government tenant add `-Environment USGov` and, if the Graph path is unavailable, pass the Agent Registry CSV export with `-RegistryCsv .\ExportedAgents.csv`. The script flags agents whose `AvailableTo` = `all` (shared org-wide), agents shared to large groups, and agents lacking an owner.

1. For each flagged agent, the register custodian opens a review item and routes it to the Data &

Security Reviewer. The outcome is one of: **keep** (owner + sponsor named, sharing justified, recorded in the register); **narrow** (owner reduces the sharing scope to the reviewed set); or **retire** (no owner or no justification — go to Step 10).

1. Record every outcome in a list named exactly `AGBGOV – Shared-Agent Access Review` with columns:

`Agent Name`, `Agent Id`, `Current Sharing Scope`, `Reviewer`, `Review Date`, `Outcome` (`Keep` / `Narrow` / `Retire`), `Action Taken`, `Decision Reference`.

1. Set the **review cadence**: broadly-shared and org-published agents are re-reviewed **every 6 months**; the sweep in step 2 runs** monthly** so newly broad-shared agents surface quickly.

Step 10 — Define the lifecycle and retirement process

Agents outlive their makers. Retirement is the step that actually shrinks risk, and it must be explicit and evidenced.

1. Adopt the lifecycle states already in the register: `Requested → In review → Approved – pilot → Approved – business use → Deprecated → Retired`.

1. Define the **retirement triggers**: the Agent Owner has left and the agent is ownerless; the agent is superseded by another; the agent is no longer used; a review lapsed and was not renewed; or a security/compliance concern requires it.

1. Define the **retirement procedure**, and follow it for every retirement:

• The Business Sponsor (or, for ownerless agents, the register custodian) confirms retirement and records an `AGBGOV – Agent Decision Record` entry.

• The Agent Admin **blocks** the agent first (Microsoft 365 admin center → **Agents → All agents → Registry → **open the agent** → **flyout** → Block**). Blocking is reversible and is the safe pause.

• Communicate a **notice period** to known users where the agent is in active use (default 30 days); ownerless or unused agents may be blocked immediately.

• After the notice period with no valid objection, **delete** the agent. For an agent the maker built, only the maker can delete it in Agent Builder; for ownerless agents the Agent Admin deletes it from the Agent Registry. *Deletion is permanent and irreversible* — the block-then- wait sequence exists precisely so deletion is never the first action.

• Record the block date, delete date, and evidence link in the `AGBGOV – Agent Retirement Log`.

1. For **ownerless agents specifically**, use the admin center's built-in handling: the **Agents without owners **card filters the Agent Registry to ownerless agents in one click; from there the Agent Admin either** reassigns** a new owner (if the agent should live on) or blocks/deletes it (if not). Reassign rather than orphan whenever a business use still exists.

Step 11 — Stamp governance onto the existing estate

Now bring the agents that already exist under the model.

1. Open `AgentEstate-Baseline.csv` from Step 1. For every declarative agent, decide its disposition:

ratify (a real business use with an owner you can name), **narrow/keep** (per the Step 9 review), or **retire** (Step 10).

1. For each agent to ratify, create or complete its `AGBGOV – Agent Register` item: name the Agent Owner, Business Sponsor, and (if sensitive) Data & Security Reviewer; set `Lifecycle State` to `Approved – business use`; and record a bulk-ratification reference `AGBARB-<YYYY>-LEGACY-<NNN>` in the `AGBGOV – Agent Decision Record`.

1. Rename any agent that does not conform to the Step 6 naming standard. The Agent Owner edits the name in Agent Builder (**All agents** → **…** → **Edit** → update name → **Update**), or the admin updates it via the Agent Registry. Renaming does not change who the agent is shared with.

1. For each agent to retire, run the Step 10 procedure.

2. Re-run the validation script to confirm the estate now largely conforms:

```
powershell -ExecutionPolicy Bypass -File .\AGBGOV-Validation.ps1 -RegisterCsv .\AGBGOV-Register-Export.
```

(Export the `AGBGOV – Agent Register` list to CSV first, or point the script at the Agent Registry export — it accepts either.)

Step 12 — Pilot the model for two full board cycles

Do not roll the model across the whole tenant immediately. Run it narrow and prove it.

1. Choose the pilot scope: **one business function** (for example, Finance) **or one agent family** (for example, all HR-facing agents). Record the choice.

1. Run the model exactly as documented for **two complete ARB cycles** — at a fortnightly cadence, four weeks.

1. During the pilot, route **every** new or changed agent in scope through the intake form. No exceptions, including urgent ones — an urgent request is a Tier 3/Tier 4 change, not a bypass.

1. At the end of each cycle, run the validation and reporting scripts:

```
powershell -ExecutionPolicy Bypass -File .\AGBGOV-Validation.ps1 -RegisterCsv .\AGBGOV-Register-Export.
powershell -ExecutionPolicy Bypass -File .\AGBGOV-Report.ps1 -RegisterCsv .\AGBGOV-Register-Export.csv
```

1. Hold a 30-minute retrospective after cycle 2 with the board. Ask: which gate produced the most rework, which evidence requirement was hardest to supply, and where did someone attempt to share an agent around the gate? Amend the standards document and increment its version.

1. Only when Section 5's gates pass, proceed to Step 13.

Step 13 — Expand to the whole tenant and enter steady state

1. Make the intake form the **only** sanctioned route for a new business agent tenant-wide. Announce the date, and have the executive sponsor from R1 send the announcement — not IT.

1. Tighten the Step 5b sharing control from `AGBGOV – Agent Sharing – Org-Wide Approved` to the final posture your pilot proved sustainable (specific approved group, or no org-wide sharing at all).

1. Bulk-ratify or retire the remaining legacy agents: ratify those with a nameable owner and business use; move the rest to `Deprecated`, give the 30-day notice, and retire. This is the step that shrinks the estate.

1. Establish the steady-state cadence:

Activity	Frequency	Owner
Agent Review Board	Fortnightly	Register custodian (chair)
Governance scorecard published (`AGBGOV-Report.ps1`)	Monthly	Register custodian
Conformance validation (`AGBGOV-Validation.ps1`)	Monthly	Agent Admin
Shared-agent sweep (`AGBGOV-Review-SharedAgents.ps1`)	Monthly	Data & Security Reviewer
"Agents at risk" / ownerless review in the admin center	Monthly	Agent Admin
Broadly-shared / published-agent access review	Every 6 months	Data & Security Reviewer
Standards & change-classification review	Annually	Executive sponsor + Security

1. Add the scorecard to an existing governance forum's standing agenda (an IT operations review, a data-governance board, or an AI council). Metrics reported nowhere change nothing.

1. Archive the Step 1 baseline alongside the first steady-state scorecard. The delta — fewer ownerless agents, fewer unreviewed org-wide shares, more agents with named owners — is the business case for having done this, and it is what you present at the annual review.

Testing and Validation

The point of testing is not to confirm the documents exist — it is to prove the **governance gates actually function**: that an agent cannot reach business use without an owner and an approval, that an unreviewed org-wide share is caught, and that a stale or ownerless agent is retired. Each gate below has an explicit **PASS / FAIL** criterion. Do not proceed to Step 13 (expand) until every gate passes for the pilot scope.

Use the exact object names from Section 3 throughout.

Gate 1 — Build eligibility is enforced

What you are proving: only members of `AGBGOV – Agent Builders` can create agents (where your policy restricts build).

Procedure

1. Sign in as a test user who is **not** in `AGBGOV – Agent Builders`.
2. Open Microsoft 365 Copilot → **Agents** → **New agent / Agent Builder**.

PASS: the test user cannot create an agent (Agent Builder is unavailable or blocked), consistent with the Copilot Control System agent access setting from Step 5a. **FAIL:** the unsanctioned user can build and share an agent. Re-check Step 5a — confirm **Specific users/groups** is set to `AGBGOV – Agent Builders` and that the change has propagated (allow time for tenant replication).

> *If your policy is deliberately **open build** with post-hoc governance, mark this gate **N/A** and record that decision — but Gate 2 and Gate 3 then carry the whole load.*

Gate 2 — Org-wide sharing is restricted

What you are proving: an ordinary user cannot share an agent with **"Anyone in your organization"** — the core gap this product closes.

Procedure

1. Sign in as a sanctioned builder who is **not** in `AGBGOV – Agent Sharing – Org-Wide Approved`.
2. Build a trivial test agent named `AGBGOV – Test – Sharing Gate – Pilot`.
3. Select **Share** and inspect the sharing options.

PASS: the **Anyone in your organization** option is **greyed out** (with the restriction tooltip) and only **Only you** / **Specific users in your organization** are available. **FAIL:** the user can select **Anyone in your organization**. Re-check Step 5b — confirm the agent sharing control is set to **Specific users or groups** = `AGBGOV – Agent Sharing – Org-Wide Approved`, and remember the documented behaviour that the control affects **new** sharing actions only.

Gate 3 — The intake / approval gate holds

What you are proving: an agent cannot reach `Approved – business use` without passing the ARB.

Procedure

1. Submit the test agent through the `AGBGOV – Agent Intake Request Form`.
2. Attempt to record it as `Approved – business use` in the register **without** an

`AGBGOV – Agent Decision Record` entry.

1. Run the validation script against the register export:

```
powershell -ExecutionPolicy Bypass -File .\AGBGOV-Validation.ps1 -RegisterCsv .\AGBGOV-Register-Export.
```

PASS: `AGBGOV-Validation.ps1` reports **G3_ApprovalRecorded = FAIL** for that agent (approval claimed with no `AGBARB-` reference), catching the bypass. **FAIL:** the script reports PASS despite the missing decision record. Confirm the register's `Lifecycle State` and `ARB Decision Reference` columns are populated as designed and that the export carries them.

Gate 4 — The org-catalog publish path requires admin approval

What you are proving: submitting an agent to the org catalog routes to an admin, and publication is an admin action.

Procedure

1. As a sanctioned builder, **submit the test agent to your org catalog**.
2. As the Agent Admin, open Microsoft 365 admin center → **Agents** → **All agents** → **Registry** and find the pending **agent request**.

PASS: the agent appears as a pending request and does **not** show in **Built by your org** in the Agent Store until the admin publishes it; the admin can **publish** or **reject** it. **FAIL:** the agent appears in **Built by your org** with no admin action. Re-check Step 5c and confirm the approval path is active (Government tenants: confirm the request queue is visible — publishing to the org is supported in GCCH/GCCM).

Gate 5 — Sensitive-data agents require a data review

What you are proving: an agent over `Confidential`/`Restricted` sources cannot be approved without a named Data & Security Reviewer.

Procedure

1. In the register, create a test entry with `Data Classification = Confidential`, `Lifecycle State = Approved – business use`, an owner and sponsor, but **no** Data & Security Reviewer.
1. Run `AGBGOV-Validation.ps1` as in Gate 3.

PASS: the script reports **G4_DataReview = FAIL** for that agent. **FAIL:** the script reports PASS. Confirm the `Data & Security Reviewer` and `Data Classification` columns are present and correctly named in the export.

Gate 6 — Broadly-shared and ownerless agents are surfaced

What you are proving: the monthly sweep finds the agents that need review.

Procedure

1. Ensure at least one test agent is shared org-wide and at least one has no owner (or use the real baseline).
1. Run:

```
powershell -ExecutionPolicy Bypass -File .\AGBGOV-Review-SharedAgents.ps1 -CsvPath .\SharedReview-Gate6
```

(Government / no-Graph: add `-RegistryCsv .\ExportedAgents.csv -Environment USGov`.)

PASS: the output CSV lists the org-wide agent with `Reason = OrgWideShare` and the ownerless agent with `Reason = NoOwner` and `RecommendedAction = Retire-or-Reassign`. **FAIL:** a known broadly-shared or ownerless agent is missing. Confirm the input source (Graph or Registry export) actually contains the sharing/owner columns.

Gate 7 — Retirement is real and evidenced

What you are proving: an agent can be blocked, notice given, deleted, and the evidence retained.

Procedure

1. Run the Step 10 retirement procedure on the test agent: block it in the Agent Registry, record the decision, then (after confirming) delete it.
1. Confirm the `AGBGOV – Agent Retirement Log` has an entry with block date, delete date, and reason.
2. Confirm the agent no longer appears to end users and is gone from the Agent Registry.

PASS: the agent is blocked/deleted, the retirement log entry exists, and end users can no longer invoke it. **FAIL:** the agent still runs, or no log entry exists. Blocking is reversible — re-block and complete the log before deletion. Remember deletion is **permanent**; the block-then-wait sequence is mandatory.

Gate 8 — The governance scorecard is trending the right way

What you are proving: the model is measurably improving coverage and reducing risk.

Procedure

1. Run `AGBGOV-Report.ps1` at the end of pilot cycle 1 and again at cycle 2.
2. Compare the two scorecard rows.

PASS: across the two cycles, `PctWithOwner`, `PctWithApproval`, and `PctWithReviewDate` rise, and `OwnerlessAgents`, `OverdueReviews`, and `SensitiveAgentsNoReviewer` fall (or are zero) for the pilot scope. **FAIL:** the numbers are flat or worse. Hold the Step 12 retrospective, find which gate produced the rework, amend the standards, and run another cycle before expanding.

Exit criteria for the pilot

All of Gates 1–8 **PASS** (or are explicitly recorded **N/A** with a documented reason), the pilot scope shows a positive scorecard trend, and the retrospective actions are folded into the standards document (version incremented). Only then run Step 13 (expand to the whole tenant).

Troubleshooting

Agent governance fails in two ways: **organisationally** (people route around the model) and **technically** (a control or script does not behave as expected). Both are covered below. Use the exact object names from

Section 3 when you follow a fix.

Organisational failure modes

Users are sharing agents around the gate

Symptom. New agents keep appearing shared to teams or org-wide without an intake request or an `AGBGOV – Agent Decision Record`.

Cause. Either the Step 5b org-wide sharing control is still set to **All users**, or building is open and users share via **Specific users in your organization** (which the org-wide control does not restrict), or the model was announced by IT and reads as optional.

Fix. Confirm Step 5b is set to **Specific users or groups** = `AGBGOV – Agent Sharing – Org-Wide Approved` (this only blocks *new* org-wide shares — run `AGBGOV-Review-SharedAgents.ps1` to catch existing ones). For team-level sharing that bypasses review, tighten the intake expectation and have the **executive sponsor** re-announce the gate as mandatory. Publish the Tier 1 pre-approval rule prominently so the fast path is obvious — people bypass slow gates, not fast ones.

The Agent Review Board has become a bottleneck

Symptom. The intake queue is growing; makers complain the ARB is slow; shadow agents rise.

Cause. The Tier 1 standard-change pre-approval is not being used, or the board cadence is monthly, or the evidence pack requirement is being applied to trivial changes.

Fix. Enforce the Tier 1 rule (minor instruction tweaks with no source/data/sharing change need no ARB slot). Keep the cadence fortnightly. Reserve the full evidence pack for Tier 2+ items. If volume is still high, the intake gate in Step 7 is too loose — return incomplete requests rather than tabling them.

Nobody will own the ownerless agents

Symptom. The **Agents without owners** count does not fall; agents sit ownerless for months.

Cause. No forcing function, and no sponsor willing to say "reassign or retire".

Fix. Put the ownerless count on the monthly scorecard the sponsor sees. Set a rule: an agent ownerless for 30 days is **blocked**; ownerless for 60 days with no reassignment is **retired** per Step 10. Reassign wherever a business use persists — orphaning is not a decision.

A rejected agent's maker escalates over the ARB's head

Symptom. A director insists their team's agent be shared org-wide despite a rejection or a data finding.

Cause. The decision rights are not backed by the sponsor.

Fix. This is exactly why R1 (the executive sponsor) is mandatory. Route the escalation to the sponsor, not the ARB chair. If the sponsor overrides, record it as a **documented risk acceptance** in the `AGBGOV – Agent Decision Record` with the sponsor named — governance survives an override that is recorded; it does not survive one that is hidden.

Technical failure modes

`AGBGOV-Discovery.ps1` *returns "copilotPackages API not available / not authorized" (HTTP 403 / 404)*

Symptom. The Graph path fails and the script tells you to use the CSV export.

Cause (any of). The signed-in account lacks the `CopilotPackages.Read.All` scope or the **AI Administrator** role; the tenant has **no Microsoft Agent 365 licence** (the Package Management API requires one); or you are in a **Government tenant** (the API is **not available** in US Gov L4/L5).

Fix. Export the agent list from Microsoft 365 admin center → **Agents** → **All agents** → **Registry** → **Export**, then re-run with `-RegistryCsv .\ExportedAgents.csv`. This is the expected, supported path for Government tenants and any tenant without Agent 365. *Verify current API availability against Microsoft documentation — it is preview/licence-gated and moving.*

The "Anyone in your organization" sharing option is still available after Step 5b

Symptom. A non-approved user can still share org-wide in Agent Builder.

Cause. The control change has not propagated, the wrong group was selected, or the user was already in the approved group.

Fix. Re-open the Copilot Control System agent sharing setting; confirm **Specific users or groups** = `AGBGOV – Agent Sharing – Org-Wide Approved` and that the test user is not a member. Allow time for tenant replication. Remember the documented behaviour: the control governs **new** sharing actions and does **not** retroactively revoke existing org-wide shares — clean those up via `AGBGOV-Review-SharedAgents.ps1` and Step 9.

An agent shared with a user still returns no answers from its knowledge

Symptom. The agent is shared, but the recipient gets empty or partial responses.

Cause. The agent respects the **end user's** information and sensitivity privileges — if the recipient lacks access to the underlying SharePoint/OneDrive knowledge source, the agent excludes that content. Automatic SharePoint file/folder sharing only occurs when the agent is shared to **Specific users in your organization** and the owner has rights to share those files; sensitivity labels are honoured.

Fix. This is by design, not a governance defect — the agent cannot grant access the user does not have. If the recipient legitimately needs the data, grant access to the source in SharePoint directly (or reshare the agent's knowledge sources). Do **not** widen the source's permissions broadly to make an agent work — that recreates the oversharing this whole programme exists to control.

`AGBGOV-Validation.ps1` *reports FAIL for agents you believe are compliant*

Symptom. Gates fail unexpectedly (commonly G3 approval or G6 naming).

Cause. The CSV export uses different column headers than the script's defaults, or the naming uses a hyphen (`-`) instead of the en dash (–), or `Lifecycle State` / `ARB Decision Reference` were not exported.

Fix. The script matches column names defensively, but confirm the register export carries `Agent Owner`, `Business Sponsor`, `Data Classification`, `Sharing Scope`, `Lifecycle State`, `ARB Decision Reference`, and `Next Review Date`. Fix names to use the **en dash with single spaces** per the Step 6 standard — G6 tests exactly `AGBGOV – <Function> – <Purpose> – <Audience>`.

An agent submitted to the org catalog never reaches an admin for approval

Symptom. Makers submit to the org catalog but no request appears in the admin center.

Cause. The submitter's tenant/licensing does not support org-catalog publishing, the request queue is being viewed by an account without the right role, or (Government) you assumed a Commercial-only behaviour.

Fix. Confirm you are viewing **Agents** → **All agents** → **Registry** (agent requests) as an **AI Administrator**. Confirm publishing to the org is supported for your tenant — it **is** supported in GCC High (GCCH) and GCC Moderate (GCCM). If submissions genuinely do not appear, verify the maker used the org-catalog submission (not a direct share) and check current tenant support against Microsoft documentation.

Deleting an agent did not remove it for the users it was shared with

Symptom. After deletion, some access or knowledge-source sharing appears to persist.

Cause. Only the maker can delete a maker-built agent; when a user's access to an agent is removed, it does **not** automatically remove their access to files/folders that were shared via the agent — those are managed directly in SharePoint.

Fix. Ensure the actual owner performed the deletion (or the Agent Admin deleted an ownerless agent from the Agent Registry). Then reconcile any lingering SharePoint file/folder permissions in SharePoint itself. Record the block date, delete date, and this reconciliation in the `AGBGOV – Agent Retirement Log`.

`AGBGOV-Export-ApprovalEvidence.ps1` *produces an empty audit CSV*

Symptom. The pack builds but the directory-audit file is empty or absent.

Cause. The directory audit does not carry the specific Copilot/agent activity you need, or the account lacks `AuditLog.Read.All`.

Fix. This is expected — the authoritative trail for Copilot/agent approve/publish/block actions is the **unified audit log in Microsoft Purview** (Purview portal → **Audit**, or `Search-UnifiedAuditLog`). Export the relevant activity there and add it to the pack. The decision record remains the primary governance evidence regardless. *Verify current audit record availability for agent actions against Microsoft documentation.*

Government tenant: a documented agent feature is missing

Symptom. A capability described for Commercial (e.g. declarative agents with actions) is not present in your GCC High / DoD tenant.

Cause. Government agent features roll out on a **later, narrower schedule** than Commercial.

Fix. Do not treat this as a defect. The governance *operating model* is identical; only feature availability differs. Verify the capability's current GCC High / DoD availability against Microsoft's US-sovereign-cloud documentation and adjust scope accordingly — govern the features you actually have.

Best Practices

These are the practices that separate an agent governance model that *works* from one that becomes a document nobody reads. They are drawn from the same operating-model discipline used for application governance, adapted to the specific realities of business-user-built AI agents.

Govern the sharing, not the building

The instinct is to lock down who can build agents. Resist over-tightening it. The productivity value of Agent Builder comes from letting business users build; the *risk* comes from where those agents go and what they reach. Put your strongest control on **org-wide sharing** and on the **org-catalog publish gate**, and let building stay relatively open inside `AGBGOV – Agent Builders`. A model that bans building will simply be bypassed; a model that reviews *sharing* is both safer and more welcome.

Blocking is free — use it early, unblock later

The single most important operational habit: **any Agent Admin may block any agent immediately** on a credible concern, with no meeting and no approval. Blocking is reversible. Treat it like a circuit breaker, not a punishment. The organisations that get burned are the ones that require a change request to block — turning a two-minute containment into a multi-hour exposure. Make "block first, investigate second" an explicit, celebrated norm.

Never delete as the first action

Deleting an agent is **permanent and irreversible**, and for maker-built agents only the maker can do it. Always **block** → **notice period** → **delete**. The block is the safety; the notice period catches the "actually, Finance still uses that every month" objection before it becomes an incident. Reserve immediate deletion for agents that are both ownerless and demonstrably unused.

Reassign before you orphan

When a maker leaves, their agents become ownerless — but many still deliver real value. Use the admin center's **Agents without owners** view to triage: **reassign** a new owner where a business use persists, and only retire the rest. An ownerless agent that is quietly reassigned keeps working; an ownerless agent that is ignored becomes a shadow-agent risk.

Make the register the source of truth for *accountability*, not inventory

Do not try to make the `AGBGOV – Agent Register` mirror every field the Agent Registry already holds. The admin center is the technical inventory; the register exists to answer *who is accountable, what was decided, and when is it next reviewed*. Keep it lean — owner, sponsor, reviewer, classification, sharing scope, decision reference, review date — and let the scripts reconcile it against the admin-center export.

Tie the data review to the data, and reuse the Purview controls you already have

An agent's risk is a function of the **sensitivity of the sources it reads** and **how widely it is shared**. Trigger the Data & Security Reviewer only when those cross a threshold (Confidential/ Restricted data, or org-wide sharing). Where a review finds a real data-processing concern, hand it to the Copilot data-governance / DLP solution's controls (DLP for Microsoft 365 Copilot, sensitivity labels) rather than inventing a new control here. Governance that reuses existing enforcement is governance that sticks.

Keep the ARB fast and cheap

A fortnightly, 45-minute board with a hard Tier 1 pre-approval rule is sustainable. A weekly two-hour board is not, and a monthly board makes IT the bottleneck that drives users to share agents around the gate. The

Tier 1 standard-change rule (minor instruction tweaks, no source/data/sharing change) typically removes the majority of the board's volume — protect it.

Watch the "Agents at risk" signals — but don't rebuild those platforms

The Microsoft 365 admin center aggregates agent risk from Entra, Purview, and Defender — shadow agents, ownerless agents, excessive-permission agents, sensitive-data-access, pending-approval. Put a monthly review of that card on the Agent Admin's cadence. Your job in this model is to define *who acts and when*; the detection is Microsoft's platforms' job. Do not attempt to reproduce those signals in your own tooling.

Treat review intervals shorter for agents than for apps

Agents drift faster than packaged applications — instructions get edited, sources get added, sharing gets widened, all by business users, often without telling anyone. Do not exceed a 12-month review interval for any tier, and review Tier 4 (regulated/critical) every 6 months. A lapsed review moves the agent to `Deprecated` automatically.

Announce the gate from the business, not from IT

When you make the intake form the only sanctioned route and tighten org-wide sharing, have the **executive sponsor** send the announcement. A governance change announced by IT reads as a restriction; the same change announced by the business sponsor reads as a standard. Adoption of the model depends on this more than on any technical control.

Government tenants: verify feature availability before you promise it

The operating model is identical in GCC / GCC High / DoD, but the **agent feature set is narrower and evolving** and the `copilotPackages` Graph API is unavailable there. Seed and sweep from the Agent Registry CSV export, and never promise a specific agent capability in a Government tenant without first verifying its current availability against Microsoft's US-sovereign-cloud documentation.

Deliverables Summary

This solution is a complete, self-contained operating model for governing business-user-built declarative agents (Agent Builder / Copilot Studio / SharePoint agents). Everything below is included in the package.

Documentation

Document	What it gives you
Solution Overview	The problem, the explicit scope boundary (what this is and is NOT), the seven components of the model, how declarative agents are built/shared/approved/managed (verified against Microsoft Learn), how it plugs into Purview/Entra/Defender, and the Government note.

Document	What it gives you
Requirements	Ten prerequisites (sponsor, licensing, the AI Administrator role, Entra group rights, system of record, data-governance posture, CAB interface, role-holders, Graph PowerShell, build-surface scope) — each with Description / Why / Verify / Implement-if-missing and exact roles/scopes.
Implementation	Thirteen ordered steps that stand up the whole model: baseline the estate, define the RACI, build the register/decision-record/retirement-log, create the six governance groups, set the Copilot Control System enforcement controls, publish the standards, stand up intake and the Agent Review Board with change classification, set the sharing policy and access review, define lifecycle and retirement, stamp the existing estate, pilot, and expand.
Testing & Validation	Eight PASS/FAIL governance gates that prove the model *functions* (build eligibility, org-wide sharing restriction, intake/approval, org-catalog admin approval, sensitive-data review, broadly-shared/ownerless detection, real retirement, scorecard trend) plus pilot exit criteria.
Troubleshooting	Organisational and technical failure modes with symptoms, causes, and fixes — from "users share agents around the gate" to "the copilotPackages API returns 403".
Best Practices	The habits that make the model stick: govern sharing not building, block early, never delete first, reassign before orphaning, keep the ARB fast, reuse existing Purview controls, shorter review intervals for agents.
Executive Summary	A one-page brief for the sponsor and stakeholders.

Governance artefacts (pre-named, ready to create)

- `AGBGOV – Agent Register` (SharePoint list)
- `AGBGOV – Agent Decision Record` (SharePoint list)
- `AGBGOV – Agent Retirement Log` (SharePoint list)
- `AGBGOV – Shared-Agent Access Review` (SharePoint list)
- `AGBGOV – Agent Governance RACI – Enterprise – v1.0`
- `AGBGOV – Agent Standards – Enterprise – v1.0`
- `AGBGOV – Agent Change Classification – Enterprise – v1.0`

- `AGBGOV – Agent Intake Request Form` + `AGBGOV – Agent Intake Queue`
- `AGBGOV – Agent Review Board – Enterprise – Fortnightly`
- Six Entra security groups: `AGBGOV – Agent Builders`, `AGBGOV – Agent Sharing – Org-Wide Approved`,

`AGBGOV – Agent Owners`, `AGBGOV – Agent Business Sponsors`, `AGBGOV – Agent Data-Security Reviewers`, `AGBGOV – Agent Admins`

PowerShell scripts (Microsoft Graph, read-only, cloud-aware)

All five accept `-Environment Global | USGov | USGovDoD` and are read-only by default; any Graph call that is preview/licence-gated degrades gracefully to the Agent Registry CSV export.

Script	Purpose
AGBGOV-Discovery.ps1	Governance baseline: current declarative agents, owners, and sharing posture — seeds the register. Graph `copilotPackages` path with automatic CSV-export fallback.
AGBGOV-Validation.ps1	Tests each agent against the seven governance gates (owner, sponsor, approval, data review, sharing justification, naming, review currency); PASS/FAIL per gate; optional pipeline exit code.
AGBGOV-Report.ps1	Trendable governance scorecard (coverage %, sharing exposure, ownerless, overdue reviews, sensitive-without-reviewer, optional intake-to-decision time).
AGBGOV-Export-ApprovalEvidence.ps1	Assembles a timestamped, audit-ready approval evidence pack from the decision record plus (optional) directory audit trail.
AGBGOV-Review-SharedAgents.ps1	Flags org-wide, large-group, and ownerless agents for periodic access review with recommended actions.

Scope boundary (what is intentionally delivered elsewhere)

- The **deep, repeatable agent inventory / sharing export and the Graph endpoint diagnostic** →

sibling product *Copilot Agent Builder — Inventory, Sharing Governance & Export*.

- **Copilot data governance** (sensitivity labels, DLP for Microsoft 365 Copilot, audit/eDiscovery) →

the Copilot data-governance / DLP guardrails solution.

- **SharePoint / OneDrive oversharing remediation** → the Copilot oversharing / SharePoint-permissions

solution.

- **Building** agents (authoring instructions, knowledge, capabilities) → the maker's task in Agent

Builder / Copilot Studio.

This model defines *when* those are invoked and *who* is accountable — it does not reproduce them.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AGBGOV-Discovery.ps1

AGBGOV-Export-ApprovalEvidence.ps1

AGBGOV-Report.ps1

AGBGOV-Review-SharedAgents.ps1

AGBGOV-Validation.ps1